University of Geneva

Faculty of Science

Timing Attacks

Advanced Security
14X

Michel Jean Joseph Donnet

2026-05-02

Contents

1 Introduction

The cryptography goal is to ensure the confidentiality, integrity, and authenticity of information, and it has become increasingly important in the digital age with the rise of the internet and digital communication where sensitive information is frequently transmitted. To achieve these goals, various cryptographic algorithms and protocols have been developed, including symmetric and asymmetric encryption, hashing, and digital signatures. These algorithms are based on mathematical problems that are computationally difficult to solve, such as factoring large integers or computing discrete logarithms, which provide the basis for their security since they are believed to be infeasible to break with current computational resources.

However, although these algorithms are designed to be secure against direct attacks, the implementation of these algorithms can introduce vulnerabilities that can be exploited by attackers. The vulnerabilities reside in the way the algorithms are implemented and executed, rather than in the algorithms themselves. Indeed, attackers can exploit side channels, which are unintended information leaks that occur during the execution of cryptographic algorithms such as timing information, power consumption, electromagnetic emissions, or even sound. By analyzing these side channels, attackers can gain insights into the internal workings of the cryptographic algorithm and potentially recover sensitive information such as secret keys.

In this report, we will focus on timing attacks, which are a type of side-channel attack that exploits the time it takes for a cryptographic algorithm to execute. Timing attacks can be used to recover secret keys or other sensitive information by measuring the time it takes for a cryptographic operation to complete and analyzing the variations in timing based on different inputs or conditions.

2 Timing attacks on RSA

RSA is a widely used asymmetric encryption algorithm that relies on the difficulty of factoring large integers to provide security. It allows in particular for sharing a symmetric key securely over an insecure channel, enabling secure communication between parties without the need for a pre-shared secret key.

The base principle of RSA resides in the use of a pair of keys: a public key for encryption and a private key for decryption. The RSA algorithm consists of three main steps: key generation, encryption, and decryption.

2.1 RSA basics

The key generation process involves selecting two large prime numbers, p and q, and computing their product $n = p \times q$, which serves as the modulus for both the public and private keys. The choice of p and q is crucial for the security of RSA, as the difficulty of factoring n into its prime factors is what provides the security of the algorithm. Indeed, if an attacker can factor n into p and q, they can compute the private key and break the encryption.

The algorithm of key generation can be summarized as follows:

1. Choose two distinct large prime numbers p and q.
2. Compute $n = p \times q$ and $\varphi(n) = (p-1)(q-1)$.
3. Choose an integer e such that
   - $1 < e < \varphi(n)$
   - $\gcd(e, \varphi(n)) = 1$
4. Compute d such that $e \cdot d \equiv 1 \bmod \varphi(n)$.

The public key consists of the pair (n, e), while the private key consists of the pair (n, d).

The encryption process consists of taking a plaintext message m and computing the ciphertext $c = m^e \bmod n$ using the public key, and the decryption process consists of taking the ciphertext c and computing the plaintext message $m = c^d \bmod n$ using the private key.

For convenience, we will use the following notations throughout the report:
- n: the modulus, which is the product of two large prime numbers p and q.
- e: the public exponent, which is an integer that is coprime to $\varphi(n)$.
- d: the private exponent, which is an integer such that $e \cdot d \equiv 1 \bmod \varphi(n)$.
- m: the plaintext message, which is an integer that is less than n.
- c: the ciphertext, which is an integer that is less than n.

2.2 Timing attacks on RSA

Before the exchange of symmetric keys, the client and the server need to perform an RSA encryption and decryption operation to establish a secure communication channel. The timing of these operations can be exploited by attackers to infer information about the private key.

As mentioned earlier, the security of RSA relies on the difficulty of factoring large integers, but the implementation of RSA can introduce vulnerabilities that can be exploited by attackers. The goal of the attacker is to recover the unknown private key d thanks to the knowledge of the public key (n, e) and the ability to perform encryption and decryption operations on the server using the RSA algorithm.

2.2.1 Square-and-multiply algorithm

Timing attacks on RSA exploit the fact that the time it takes to perform certain operations in the RSA algorithm can vary based on the input values and the internal state of the algorithm. For example, the time it takes to perform the modular exponentiation operation $c^d \bmod n$ can vary based on the value of d and the input ciphertext c. Indeed, a simple implementation of the modular exponentiation operation can use a square-and-multiply algorithm, as described in the following python code:

```python
def square_and_multiply(y, x, n):
  s = 1
  y %= n

  while x > 0:
      if (x % 2) == 1:
          s = (s * y) % n # Extra multiplication here when x is odd !
```

```
        y = (y * y) % n
        x = x >> 1

    return s
```

This code computes $y^x \bmod n$ using the square-and-multiply algorithm, which is an efficient method for performing modular exponentiation. Indeed, computing $y^x \bmod n$ for very large values of x is computationally expensive.

The base idea of this algorithm is that computing y^x can be done by taking the binary representation of x, which is basically the decomposition of x into powers of 2, and then iteratively computing $y^x \bmod n$ by taking at each step the previous result and multiplying it by y if the current bit of x is 1, and then squaring the input y.

For instance, suppose we want to compute $6^{13} \bmod 17$.

- The binary representation of 13 is 1101, which corresponds to the powers of 2: $2^3 + 2^2 + 2^0$.
- We start with $s = 1$ and $y = 6$.
- For the first bit (1), we compute
  - $s = (s \cdot y) \bmod 17 = (1 \cdot 6) \bmod 17 = 6$
  - $y = y^2 \bmod 17 = 6^2 \bmod 17 = 2$.
- For the second bit (0), we compute
  - s remains unchanged since the bit is 0, so $s = 6$.
  - $y = y^2 \bmod 17 = 2^2 \bmod 17 = 4$. Note that here, y^2 corresponds to $y^{2^2} = y^{2 \cdot 2} = y^4$.
- For the third bit (1), we compute
  - $s = (s \cdot y) \bmod 17 = (6 \cdot 4) \bmod 17 = 24 \bmod 17 = 7$
  - $y = y^2 \bmod 17 = 4^2 \bmod 17 = 16$.
- For the fourth bit (1), we compute
  - $s = (s \cdot y) \bmod 17 = (7 \cdot 16) \bmod 17 = 112 \bmod 17 = 15$
  - $y = y^2 \bmod 17 = 16^2 \bmod 17 = 256 \bmod 17 = 1$.

So the result of $6^{13} \bmod 17$ is 15. The efficiency of the algorithm comes from the fact that it iteratively computes $y^x \bmod n$ by performing small multiplication and squaring operations thanks to the modular properties instead of performing a single large exponentiation operation.

But the timing of this algorithm can vary based on the value of the exponent x and the input y. Indeed, as seen in the above example, the algorithm performs an extra multiplication only when the bit of x is 1, which can lead to a longer execution time compared to when the bit is 0. This variation in timing is the basis for timing attacks on RSA.

2.2.2 Exploiting timing variations

Attackers can exploit the timing variations in the square-and-multiply algorithm to recover the private key d by measuring the time it takes for the server to perform decryption operations. Note that the attacker must know the modulus n, which is public.

First, the attacker sends multiple random ciphertexts c to the server and measures the time it takes for the server to perform the decryption operation $c^d \bmod n$.

Then for each bit of the private key d, the attacker guesses the value of the bit (0 or 1) and computes the expected timing of the decryption operation based on the guess for all the ciphertexts c.

Finally, the attacker subtracts his obtained timing measurements from the timing measurements obtained from the server and analyzes the variance of the obtained results. If the variance is low, it means that the guess for the bit is correct, while if the variance is high, it means that the guess for the bit is incorrect.

By repeating this process for all the bits of the private key d, the attacker can recover the entire private key.

2.2.2.1 Mathematical analysis

Let T_i be the time taken for the server to perform the decryption operation $c^d \bmod n$ for a given ciphertext c_i.

Let x_b be the guess for the b first bits of the private key d.

Let $t(c_i, x_b)$ be the time taken to perform the decryption operation according to the guess x_b for the b first bits of d.

If the guess x_b is correct, the timing error $T_i - t(c_i, x_b)$ will be small, indicating a good match between the observed and expected timings. We define F as the probability to observe the timing error $T_i - t(c_i, x_b)$ if the guess x_b is correct. As each timing measurement is independent, the probability of a correct guess for the b first bits of d is proportional to the product of the probabilities of observing the timing errors for all the ciphertexts c_i:

$$P(x_b) \propto \prod_{i=1}^{N} F(T_i - t(c_i, x_b))$$

So the probability a correct guess for the b first bits of d is proportional to F.